

COL362/632 Introduction to Database Management Systems

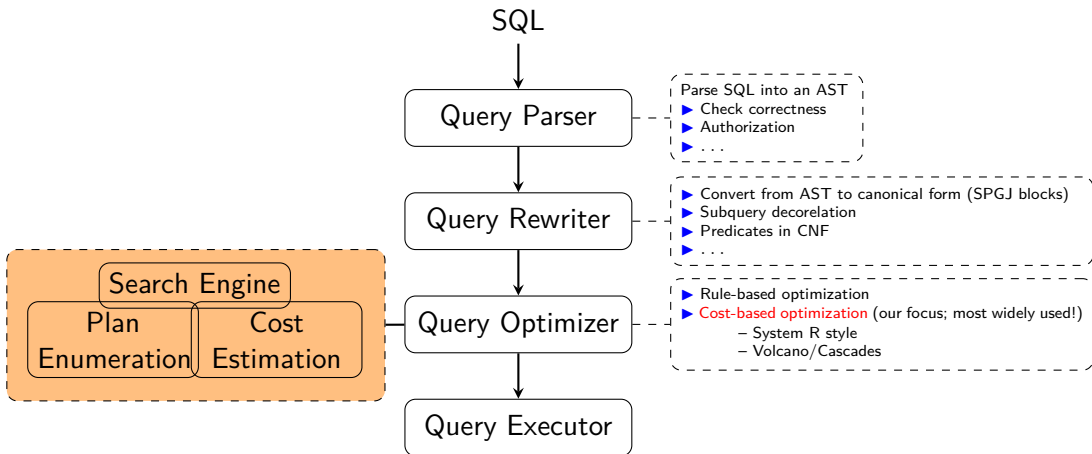
Query Optimization – Cost-based Optimization

Kaustubh Beedkar

Department of Computer Science and Engineering
Indian Institute of Technology Delhi



Query Parsing & Optimization



Cost-based Optimization

Core Idea

- ▶ Enumerate all plans
- ▶ Estimate cost of each plan
- ▶ Pick plan with least estimated cost

Seliger Optimizer

- ▶ selection pushdown
- ▶ projections up
- ▶ Optimize join orders
 - Build plans bottom up
 - Only consider left-deep trees
 - works well with existing operator implementations
 - Allows output of each operator to be pipelined into next operator
- ▶ Avoid cross products
- ▶ Use dynamic programming

1. Optimality principle must hold

- $\text{bestPlan}(R \bowtie S \bowtie T)$
= $\text{bestOf}((\text{bestPlan}(R \bowtie S) \bowtie T), (\text{bestPlan}(R \bowtie T) \bowtie S), (\text{bestPlan}(S \bowtie T) \bowtie R))$

2. Sub-problems must overlap

Dynamic Programming

```
1:  $\mathcal{R} \leftarrow \{R_1, \dots, R_n\}$ 
2: for each  $R \in \mathcal{R}$  do
3:    $\text{bestPlan}(\{R\}) \leftarrow \text{AccessPaths}(R)$ 
4:    $\text{prune}(\text{bestPlan}(\{R\}), \text{costs}())$ 
5: for  $i = 2$  to  $n$  do
6:   for  $\mathcal{S} \subset \{R_1, \dots, R_n\}$  such that  $|\mathcal{S}| = i$  do
7:      $\text{bestPlan}(\mathcal{S}) \leftarrow \emptyset$ 
8:     for each  $R \in \mathcal{S}$  do
9:        $\text{bestPlan}(\mathcal{S}) \cup = \text{bestPlan}(\mathcal{S} \setminus R) \oplus \text{bestPlan}(\{R\})$ 
10:     $\text{prune}(\text{bestPlan}(\mathcal{S}), \text{costs}())$ 
11:  $\text{prune}(\text{bestPlan}(\mathcal{R}), \text{costs}())$ 
12: return  $\text{bestPlan}(\mathcal{R})$ 
```

Dynamic Programming (Example)

DP Table				
subset				best plans
R	S	T	U	
X				scan(R), isseek(r, R), isam(a,A)
	X			scan(S), isam(a,S)
		X		scan(T), isam (c,T)
			X	scan(U), isseek(u,U), isam(c,U)

```
select R.w,S.x,T.y,U.z
```

```
from R, S, T, U, V
```

```
where
```

```
    R.a = S.a and
```

```
    S.b = T.b and
```

```
    T.c = U.c and
```

```
    R.r = 10 and U.u > 25
```

► unclustered index on *R.r*

► unclustered index on *U.u*

► clustered index on keys (a,b,c)

Dynamic Programming (Example)

DP Table				
subset				best plans
R	S	T	U	
X				scan(R), iseek(r, R) , isam(a,A)
	X			scan(S), isam(a,S)
		X		scan(T), isam(c,T)
			X	scan(U) , isam(u,U), isam(c,U)

```
select R.w,S.x,T.y,U.z
```

```
from R, S, T, U, V
```

```
where
```

```
    R.a = S.a and
```

```
    S.b = T.b and
```

```
    T.c = U.c and
```

```
    R.r = 10 and U.u > 25
```

► unclustered index on *R.r*

► unclustered index on *U.u*

► clustered index on keys (a,b,c)

Dynamic Programming (Example)

DP Table				
subset				best plans
R	S	T	U	
X				iseek(r, R)
	X			isam(a,S)
		X		isam (c,T)
			X	scan(U)
X	X			iseek(r,R) ⋈ ^{SHJ} isam(a,S), isek(r,R) ⋈ ^{BNLJ} isam(a,S), . . .
X		X		iseek(r,R) × isam(a,S), isam(a,S) × isek(r,R)
X			X	iseek(r,R) × scan(U), scan(U) × isek(r,R)
	X	X		isam(a,S) ⋈ ^{SHJ} isam(c,T), isam(c,T) ⋈ ^{SHJ} isam(a,S),...
	X		X	isam(S) × scan(U),...
		X	X	isam(c,T) ⋈ ^{SHJ} scan(U), scan(U) ⋈ ^{SHJ} isam(c,T),...

```
select R.w,S.x,T.y,U.z
from R, S, T, U, V
where
```

```
    R.a = S.a and
    S.b = T.b and
    T.c = U.c and
    R.r = 10 and U.u > 25
```

- unclustered index on *R.r*
- unclustered index on *U.u*
- clustered index on keys (a,b,c)

Dynamic Programming (Example)

DP Table				
subset				best plans
R	S	T	U	
X				iseek(r, R)
	X			isam(a,S)
		X		isam (c,T)
			X	scan(U)
X	X			$\text{iseek}(r,R) \bowtie^{SHJ} \text{isam}(a,S), \text{iseek}(r,R) \bowtie^{BNLJ} \text{isam}(a,S), \dots$
X		X		$\text{iseek}(r,R) \times \text{isam}(a,S), \text{isam}(a,S) \times \text{iseek}(r,R)$
X			X	$\text{iseek}(r,R) \times \text{scan}(U), \text{scan}(U) \times \text{iseek}(r,R)$
	X	X		$\text{isam}(a,S) \bowtie^{SHJ} \text{isam}(c,T), \text{isam}(c,T) \bowtie^{SHJ} \text{isam}(a,S), \dots$
	X		X	$\text{isam}(S) \times \text{scan}(U), \dots$
		X	X	$\text{isam}(c,T) \bowtie^{SHJ} \text{scan}(U), \text{scan}(U) \bowtie^{SHJ} \text{isam}(c,T), \dots$

```
select R.w,S.x,T.y,U.z
from R, S, T, U, V
where
```

```
  R.a = S.a and
  S.b = T.b and
  T.c = U.c and
  R.r = 10 and U.u > 25
```

- unclustered index on *R.r*
- unclustered index on *U.u*
- clustered index on keys (a,b,c)

Dynamic Programming (Example)

DP Table				
subset				best plans
R	S	T	U	
X				iseek(r, R)
	X			isam(a,S)
		X		isam (c,T)
			X	scan(U)
X	X			iseek(r,R) ⋈ ^{SHJ} isam(a,S)
X		X		iseek(r,R) × isam(a,S)
X			X	iseek(r,R) × scan(U)
	X	X		isam(c,T) ⋈ ^{SHJ} isam(a,S)
	X		X	isam(S) × scan(U)
		X	X	isam(c,T) ⋈ ^{SHJ} scan(U)
X	X	X		(iseek(r,R) ⋈ ^{SHJ} isam(a,S)) ⋈ ^{SHJ} isam(c,T)
X	X		X	(iseek(r,R) ⋈ ^{SHJ} isam(a,S)) × scan(U)
X		X	X	(isam(c,T) ⋈ ^{SHJ} scan(U)) × iseek(r,R)
	X	X	X	(isam(c,T) ⋈ ^{SHJ} isam(a,S)) ⋈ ^{SHJ} scan(U)

```
select R.w,S.x,T.y,U.z
```

```
from R, S, T, U, V
```

```
where
```

```
    R.a = S.a and
```

```
    S.b = T.b and
```

```
    T.c = U.c and
```

```
    R.r = 10 and U.u > 25
```

► unclustered index on *R.r*

► unclustered index on *U.u*

► clustered index on keys (a,b,c)

Dynamic Programming (Example)

DP Table				
subset				best plans
R	S	T	U	
X				iseek(r, R)
	X			isam(a,S)
		X		isam (c,T)
			X	scan(U)
X	X			iseek(r,R) ⋈ ^{SHJ} isam(a,S)
X		X		iseek(r,R) × isam(a,S)
X			X	iseek(r,R) × scan(U)
	X	X		isam(c,T) ⋈ ^{SHJ} isam(a,S)
	X		X	isam(S) × scan(U)
		X	X	isam(c,T) ⋈ ^{SHJ} scan(U)
X	X	X		(iseek(r,R) ⋈ ^{SHJ} isam(a,S)) ⋈ ^{SHJ} isam(c,T)
X	X		X	(iseek(r,R) ⋈ ^{SHJ} isam(a,S)) × scan(U)
X		X	X	(isam(c,T) ⋈ ^{SHJ} scan(U)) × iseek(r,R)
	X	X	X	(isam(c,T) ⋈ ^{SHJ} isam(a,S)) ⋈ ^{SHJ} scan(U)
X	X	X	X	

```
select R.w,S.x,T.y,U.z
```

```
from R, S, T, U, V
```

```
where
```

```
    R.a = S.a and
```

```
    S.b = T.b and
```

```
    T.c = U.c and
```

```
    R.r = 10 and U.u > 25
```

► unclustered index on *R.r*

► unclustered index on *U.u*

► clustered index on keys (a,b,c)

Interesting Orders

Interesting orders

- ▶ Physical property of a relation, e.g., Relation sorted on some attribute
- ▶ Breaks the principle of optimality
- ▶ Intermediate relation has an interesting order, if the order can be used later to
 - sort later (order by)
 - group by
 - perform merge join

Dynamic programming w/ Interesting Orders

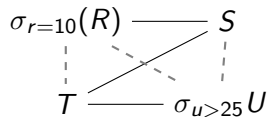
- ▶ For each subset of relations, compute multiple optimal plans (one for each interesting order)

Exploiting Query Graph Structure

Query Graph

- ▶ Undirected graph with $R_1, \dots, R_n$ as nodes
- ▶ Join predicate of the form $R_i.a = R_j.b$ forms an edge between R_i and R_j
- ▶ Predicate of the form $R_i.a = c$ are pushed down

Example



Design Principles

Focus on extensibility

1. Query processing grounded in algebraic techniques
 - Define new algebra operators, equivalence rules, operator implementation algorithms
2. Rules
3. Based on algebraic equivalences
4. Parameterized rule compilation
5. Dynamic programming based search engine (top-down approach)

Expressions

- ▶ Some query operation with zero or more input expression
e.g., logical expression $R \bowtie S$
e.g., physical expression $\text{iseek}(id, R) \bowtie^{SHJ} \text{isam}(id, S)$

Rules

1. Transformation Rules
 - E.g, $R \bowtie S \rightarrow S \bowtie R$
 2. Implementation Rules
 - E.g., $R \bowtie S \rightarrow R \bowtie^{SMJ} S$
- ▶ Each rule is specified as
 - pattern that defines the structure of the expression
 - and resulting transformation

Properties

- ▶ Logical properties
 - derived from logical algebra expression
- ▶ Physical properties
 - depend on algorithms, e.g, sort order

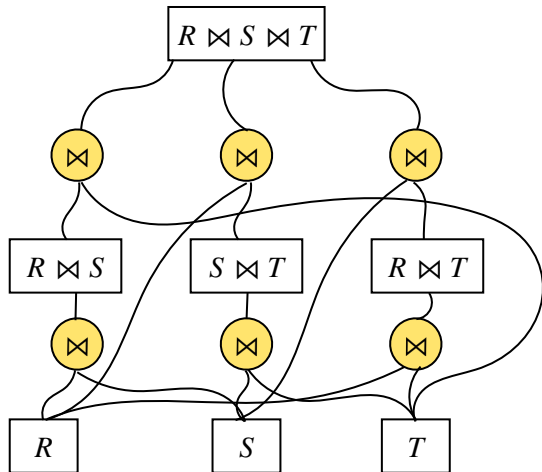
Enforcers

- ▶ Ensure property, e.g., sort, partitioning
- ▶ can also destroy properties

AND-OR DAG

Directed Acyclic Graph (DAG)

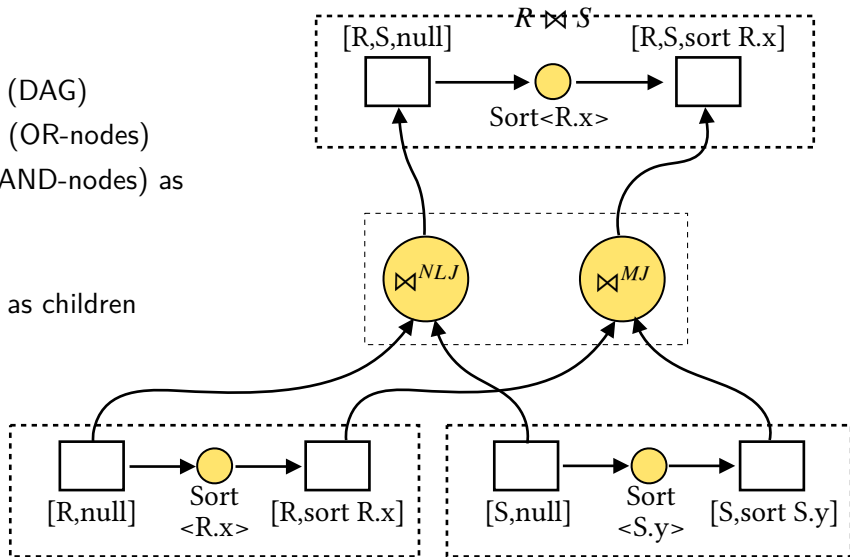
- ▶ Equivalence nodes (OR-nodes)
Operation nodes (AND-nodes) as children
- ▶ Operation nodes
Equivalence nodes as children



AND-OR DAG

Directed Acyclic Graph (DAG)

- ▶ Equivalence nodes (OR-nodes)
Operation nodes (AND-nodes) as children
- ▶ Operation nodes
Equivalence nodes as children



Finding the Best Plan

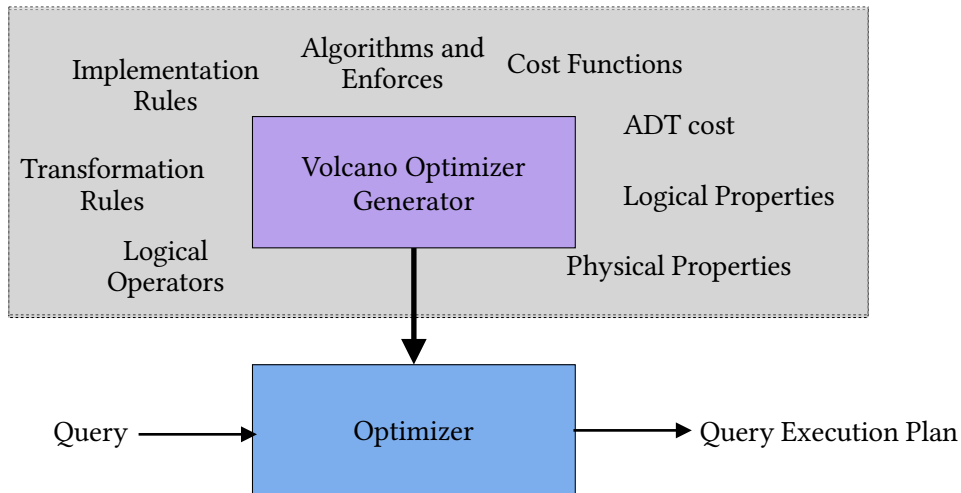
- ▶ DAG generation interleaved with finding the best plan

Directed dynamic programming

Branch and bound pruning

More than an Optimizer

Optimizer generator framework



Nested Subqueries

Example nested subquery

```
select name from supplier s
where exists (
    select * from partsupplier ps
    where s.sid = ps.sid and ps.qty > 100
)
```

- ▶ Attribute from an outer relation is used in the inner subquery
- ▶ Correlated evaluation: evaluate the outer query and invoke the inner query — can be very inefficient! Why?

Subquery de-correlation

- ▶ Turn nested query into a join, this is not always possible!
- ▶ Use semi-joins instead! (Recall semantics of semi-join operator)

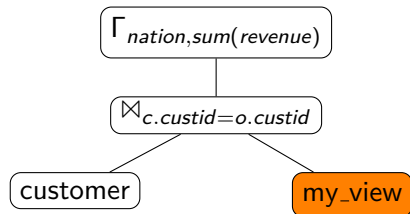
Materialized Views

- ▶ Which views to materialize?
 - similar problem: which indexes to create?
- ▶ Optimizing queries using materialized views

Example

```
create materialized view my_view as
select o.custid, count(*), sum(l.qty*o.price)
from lineitem l, orders o
where l.oid=o.oid group by o.custid
```

```
select c.nation, sum(revenue) from customer c,
(select o.custid, sum(l.qty*o.price) as revenue
from lineitem l, orders o where l.oid = o.oid
group by o.oid) as iq
where c.custid = iq.custid group by c.nation
```



Postgres Query Plans (Self Study)

- ▶ Using explain statements

<https://www.postgresql.org/docs/current/sql-explain.html>

- ▶ Explore plan Visualizer <https://explain.dalibo.com/>

- ▶ Creating Indexes

<https://www.postgresql.org/docs/current/sql-createindex.html>

- ▶ Creating statistics

<https://www.postgresql.org/docs/current/sql-createstatistics.html>